

Tutorial 8: Folds

In this tutorial will look at **folds**, first over lists, and in the **challenges** section also over trees. The function `foldr` (“r” for “right”—there is also a `foldl`) is a higher-order list function, like `filter` and `map`. It is defined as follows:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f u []      = u
foldr f u (x:xs) = f x (foldr f u xs)
```

A function `foldr f u` takes a list, and replaces every `cons ( : )` with the **folding function** `f`, and the `nil` with the **unit** `u`:

```
foldr f u (x1  : (x2  : (x3  : ...  : [] ))) ==
      x1 `f` (x2 `f` (x3 `f` ... `f` u ))
```

Many list functions are naturally given as folds:

```
sum      = foldr (+) 0
product  = foldr (*) 1
concat   = foldr (++) []
xs ++ ys = foldr (:) ys xs
```

The version `foldr1 :: (a -> a -> a) -> [a] -> a` works on non-empty lists. Instead of a unit case, it uses the last element as the basis of the fold:

```
foldr1 f (x1  : (x2  : (x3  : ...  : [xn] ))) ==
      x1 `f` (x2 `f` (x3 `f` ... `f` xn ))
```

Exercise 1:

Haskell has several more built-in functions that are straightforward folds. We will write some of them here, renamed to avoid name clashes. Write the following functions with `foldr` or `foldr1`, defining the folding function in a where-clause if needed:

- `allTrue` and `someTrue`, which behave as `and` and `or`,
- `largest` and `smallest`, which behave as `maximum` and `minimum`,
- `every` and `some`, which behave as `all` and `any`,
- `select`, which behaves as `filter`.

- e) `member`, which behaves as `elem`.
- f) `find`, which behaves as `lookup`.

Exercise 2: The following functions count certain aspects of lists, such as how often an element occurs. Write these with `foldr`.

- a) The function `evenLength` returns `True` if a list is of even length, and `False` otherwise.
- b) The function `count` returns the number of elements satisfying a given property.
- c) The function `successive` counts the longest stretch of successive occurrences of a character in a string. **Hint:** the return value of the fold should be the **current** stretch and the **longest** (previous) stretch; then after folding a further function should select the longest only.

Exercise 3: For each of the previous exercises, see if you can use `foldl` instead of `foldr`. There are three possibilities: (1) replacing `foldr` with `foldl` just works; (2) a `flip` is needed, but then the result is the same; or (3) without using `reverse`, we may get a different result with `foldl` than with `foldr`.

Challenges

Exercise 4: Give `fold` functions for the following data types. In each case, add an example tree, and use your fold function to fold it in some way.

- a) Binary trees with values at the leaves (`l`) as well as the nodes (`n`).

```
data BTree n l
  = BLeaf l
  | BNode n (BTree n l) (BTree n l)
```

- b) Binary trees with two kinds of nodes, holding two kinds of values (`a` and `b`).

```
data DTree a b
  = DLeaf
  | DNode1 a (DTree a b) (DTree a b)
  | DNode2 b (DTree a b) (DTree a b)
```

- c) Arithmetic trees for any type of number (or other value) `a`:

```
data ATree a
  = Num a
  | ATree a :+: ATree a
  | ATree a :* ATree a
  | ATree a :- ATree a
```

- d) n -Ary trees:

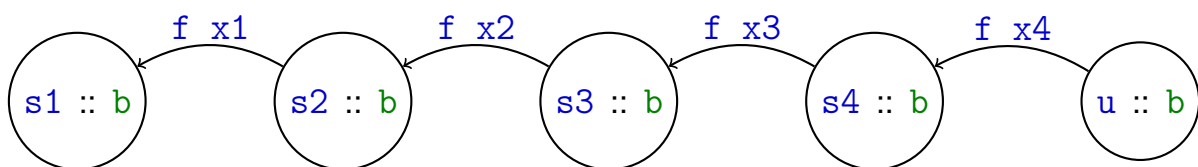
```
data NTree a = NNode a [NTree a]
```

Exercise 5: One way of viewing folds over a list is to view the result type as **states**, the list elements as (names for) **transitions**, and a **fold** as evaluating the sequential computation given by the list, making the transitions **concrete** with a specific function.

That is, the elements of a fold `foldr f u :: [a] -> b` are:

- a type of **states** `b`;
- a **starting state** `u :: b`;
- a **transition function** `f :: a -> b -> b` that gives a transition `f x :: b -> b` for any list element `x :: a`

For an input list `[x1,x2,x3,x4]` this looks as follows:



A similar picture can be made for left folds, with the arrows pointing the other direction.

Using this idea, go back to the RPN calculator tutorial, and implement the evaluation function `evaluate` of an RPN expression as a fold. Use the stack as the state, and decide whether to use a right or a left fold.